SPORE Development Task Guide

Interactive Map Feature Implementation

Project: SPORE - Soil-based Prediction Of Resident Entities
Task: Add Interactive Map for Location Selection
Estimated Time: 3-4 days
Date: December 2025

Table of Contents

1. Project Overview
2. Getting Started - Setup Instructions
3. GitHub Repository Access & Workflow
4. Task Requirements
5. Technical Specifications
6. Success Criteria
7. Troubleshooting
8. Resources

---

1. Project Overview

SPORE is a machine learning application that predicts soil microbes in India based on soil characteristics and geographic location. The system uses hierarchical classification to predict bacteria at 5 taxonomic levels (Phylum, Class, Order, Family, Genus).

Current Features:

- Dual input modes: Latitude/Longitude lookup or manual soil data entry
- Hierarchical bacterial predictions
- Professional PDF report generation

Your Task: Add an interactive map interface that allows users to select locations by clicking on a map instead of manually typing coordinates.

---

2. Getting Started - Setup Instructions

Step 1: Accept GitHub Invitation

You should have received an email invitation to collaborate on the SPORE repository. Click the link in the email or visit the repository URL to accept the invitation.

Step 2: Clone the Repository

```
git clone <repository-url>
cd SPORE
```

Step 3: Install Dependencies

```
pip install -r requirements.txt
```

Note: It is recommended to use a virtual environment:

```
python3 -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
pip install -r requirements.txt
```

Step 4: Download Pre-trained Models and Data

Download Link: https://drive.google.com/drive/folders/14iuEigwyvKQwY4NbkGE0DVLYD4jqRwH5?usp=share_link

The download contains:

- `SPORE_Models_Data.zip` (91 MB compressed, ~2GB uncompressed)

After downloading, extract the zip file:

```
unzip SPORE_Models_Data.zip
```

This creates a `SPORE_Share` folder containing:

- `models/` - Pre-trained ML models (13 files)
- `data/` - Soil database files (4 files)
- `README.txt` - Additional instructions

Step 5: Copy Files to Project

Copy the models and data folders into your SPORE project directory:

```
cp -r SPORE_Share/models/ SPORE/models/
cp -r SPORE_Share/data/ SPORE/data/
```

Your final project structure should look like:

```
SPORE/
├── app.py
├── train_model.py
├── report_generator.py
├── requirements.txt
├── templates/
│   └── index.html
├── models/
│   ├── phylum_model.pkl
│   ├── class_model.pkl
│   ├── order_model.pkl
│   ├── family_model.pkl
│   ├── genus_model.pkl
│   ├── *_encoder.pkl files
│   ├── scaler.pkl
│   ├── features.pkl
│   └── metadata.json
└── data/
    ├── HWSD2.bil
    ├── HWSD2.hdr
    ├── HWSD2_SMU.csv
    └── HWSD2_LAYERS.csv
```

Step 6: Run the Application

```
python3 app.py
```

You should see output similar to:

```
 ================================================================================
Loading Hierarchical ML Models...
================================================================================
✓ Loaded phylum model (21 classes, F1: 0.435)
✓ Loaded class model (33 classes, F1: 0.336)
✓ Loaded order model (74 classes, F1: 0.209)
✓ Loaded family model (125 classes, F1: 0.173)
✓ Loaded genus model (193 classes, F1: 0.155)
================================================================================

Loading soil database...
✓ Loaded HWSD2 raster and soil database

Starting SPORE Hierarchical Application...
```

Open your browser and navigate to: http://localhost:5002

Step 7: Verify Setup

Test the application:

1. Enter test coordinates: Latitude: 28.6139, Longitude: 77.2090 (Delhi)
2. Click "Predict"
3. Verify that bacterial predictions are displayed

IMPORTANT: Do not proceed to the development task until the application runs successfully and predictions work.

---

3. GitHub Repository Access & Workflow

Repository Access

You have been added as a collaborator to the repository with write access. The main branch is protected, which means you cannot push directly to it. All changes must go through Pull Requests.

Git Workflow

Follow this workflow for all development:

3.1 Create a Feature Branch

Always work in a feature branch, never directly in main:

```
# Ensure you're on main and it's up to date
git checkout main
git pull origin main

# Create and switch to your feature branch
git checkout -b feature/interactive-map
```

3.2 Make Your Changes

Edit the necessary files (primarily `templates/index.html`) to implement the interactive map feature.

3.3 Commit Your Changes

```
# Stage your changes
git add .

# Commit with a descriptive message
git commit -m "Add interactive map with Leaflet.js for location selection"
```

3.4 Push Your Branch

```
git push origin feature/interactive-map
```

3.5 Create a Pull Request

1. Go to the repository on GitHub
2. You should see a prompt: "Compare & pull request" - click it
3. Fill in the Pull Request description:
    - What changes you made
    - How to test the feature
    - Any issues or questions
4. Click "Create pull request"

3.6 Code Review Process

Your pull request will be reviewed. You may receive comments or requests for changes. To address feedback:

```
# Make the requested changes in your local branch
# ... edit files ...

git add .
git commit -m "Address review feedback: improve marker placement"
git push origin feature/interactive-map
```

The Pull Request will automatically update with your new commits.

3.7 After Merge

Once your PR is approved and merged:

```
# Switch back to main
git checkout main

# Pull the latest changes (including your merged code)
git pull origin main

# Delete your local feature branch (cleanup)
git branch -d feature/interactive-map
```

Important Git Rules

- DO: Always work in feature branches
- DO: Write clear, descriptive commit messages
- DO: Push your branch regularly to avoid losing work
- DO NOT: Commit directly to main branch
- DO NOT: Force push (`git push -f`) unless absolutely necessary and discussed
- DO NOT: Commit large files (models, data) - these are in .gitignore

---

4. Task Requirements

Goal

Add an interactive map to the SPORE web interface that allows users to select a location by clicking on the map, automatically filling the latitude and longitude input fields.

Functional Requirements

4.1 Map Display

- Display an interactive map on the page
- Map should be focused on India with appropriate zoom level
- India geographic bounds:
  - Latitude: 8 degrees N to 35 degrees N
  - Longitude: 68 degrees E to 97 degrees E
- Map height: approximately 400-500 pixels
- Use OpenStreetMap tiles (free, no API key required)

4.2 User Interaction

- When a user clicks anywhere on the map, capture the coordinates
- Automatically populate the latitude and longitude input fields with clicked coordinates
- Display a marker at the clicked location
- Only one marker should be visible at a time (remove previous marker when clicking new location)
- Show coordinates in a popup/tooltip when hovering over or clicking the marker

4.3 Integration with Existing Interface

- Map should appear in the "Lat/Lon Lookup" mode section
- Position the map below the coordinate input fields
- Existing functionality must continue to work:
  - Manual coordinate entry in text fields
  - Mode switching between Lat/Lon and Manual Soil Input
  - Prediction functionality
- Map should be visible/hidden appropriately based on selected input mode

4.4 Visual Design

- Map should match the existing application style
- Add appropriate borders/styling
- Ensure map is responsive and works on different screen sizes
- Consider adding a toggle button to show/hide the map (optional enhancement)

---

5. Technical Specifications

5.1 Library Selection

Research and select an appropriate JavaScript mapping library. Some popular options include:

- Leaflet.js
- Google Maps API
- Mapbox GL
- OpenLayers

Consider factors such as:

- Ease of use and learning curve
- Documentation quality
- API key requirements (free vs paid)
- Feature set needed for this project
- Community support and examples

5.2 Files to Modify

Primary file: `templates/index.html`

You will need to:

1. Add Leaflet CSS and JS links in the `<head>` section
2. Add a map container div in the appropriate location in the HTML structure
3. Add JavaScript code to initialize and handle map interactions
4. Optionally adjust CSS styling for better integration

Do not modify:

- `app.py` (backend logic)
- `train_model.py` (ML training)
- `report_generator.py` (PDF generation)
- Model or data files

5.4 Testing Considerations

Test the following scenarios:

1. Click on different locations in India - verify coordinates update correctly
2. Manually enter coordinates - map should still work
3. Switch between Lat/Lon mode and Manual Soil Input mode - ensure no JavaScript errors
4. Verify predictions still work with map-selected coordinates
5. Test on different screen sizes if possible

---

6. Success Criteria

Your implementation will be considered successful when:

- ☐ Map displays correctly on page load
- ☐ Map is centered on India with appropriate zoom level
- ☐ Clicking on map populates latitude and longitude input fields
- ☐ Marker appears at clicked location
- ☐ Only one marker is visible at a time
- ☐ Marker shows coordinates in popup
- ☐ Existing manual coordinate entry still works
- ☐ Predictions work correctly with map-selected coordinates
- ☐ No JavaScript console errors
- ☐ Code is clean and well-commented
- ☐ Pull Request includes clear description of changes

---

7. Troubleshooting

Application Won't Start

Error: "File not found" or "No such file or directory"

- Check that models/ and data/ folders are in the correct location
- Verify all files from SPORE_Share were copied correctly

Error: "No module named 'rasterio'" or similar

- Run: `pip install -r requirements.txt`
- Ensure you're in the virtual environment if you created one

Error: "Port 5002 is already in use"

- Stop any other instances of the application
- Or change the port in app.py: `app.run(debug=True, port=5003)`

Map Not Displaying

Map container is empty or shows gray box:

- Check browser console for JavaScript errors (F12 or right-click -> Inspect)
- Verify CSS and JS libraries are loading (check Network tab in browser dev tools)
- Ensure the map container div has a defined height

Map tiles not loading:

- Check internet connection (map tiles typically load from CDN)
- Verify no browser extensions blocking external content

Map doesn't resize properly:

- Check the documentation for your chosen mapping library for resize methods

Git Issues

Push rejected: "Updates were rejected because the remote contains work that you do not have locally"

```
git pull origin feature/interactive-map --rebase
git push origin feature/interactive-map
```

Cannot push to main branch:

- This is expected - main is protected
- Always work in feature branches and create Pull Requests

Merge conflicts:

```
git checkout main
git pull origin main
git checkout feature/interactive-map
git merge main
# Fix conflicts in files, then:
git add .
git commit -m "Resolve merge conflicts"
git push origin feature/interactive-map
```

---

8. Resources

Documentation

JavaScript Mapping Libraries:

- Leaflet.js: https://leafletjs.com/
- Google Maps JavaScript API: https://developers.google.com/maps/documentation/javascript
- Mapbox GL JS: https://docs.mapbox.com/mapbox-gl-js/
- OpenLayers: https://openlayers.org/

General Web Development:

- MDN Web Docs (JavaScript): https://developer.mozilla.org/en-US/docs/Web/JavaScript
- HTML DOM Events: https://developer.mozilla.org/en-US/docs/Web/Events

Flask (if needed):

- Official Documentation: https://flask.palletsprojects.com/

Getting Help

If you encounter issues:

1. Check the Troubleshooting section above
2. Review documentation for your chosen mapping library
3. Check browser console for error messages
4. Review existing code in templates/index.html for patterns
5. Ask for help - provide specific error messages and what you've tried

---

Appendix: Project Technical Stack

Backend:

- Python 3.8+
- Flask web framework
- XGBoost/Random Forest for ML models
- Rasterio for geospatial data

Frontend:

- HTML5
- CSS3
- Vanilla JavaScript
- Interactive mapping library (you will research and add)

Data:

- HWSD2 (Harmonized World Soil Database)
- GBIF bacterial occurrence data

ML Pipeline:

- Hierarchical classification at 5 taxonomic levels
- Feature engineering with soil properties
- Trained on ~80,000 bacterial occurrence records

Good luck with your implementation! Remember to ask for help if you get stuck.